

Technology Stack Comparison

Comprehensive Guide to Data Ingestion Tools & Technologies

Table of Contents

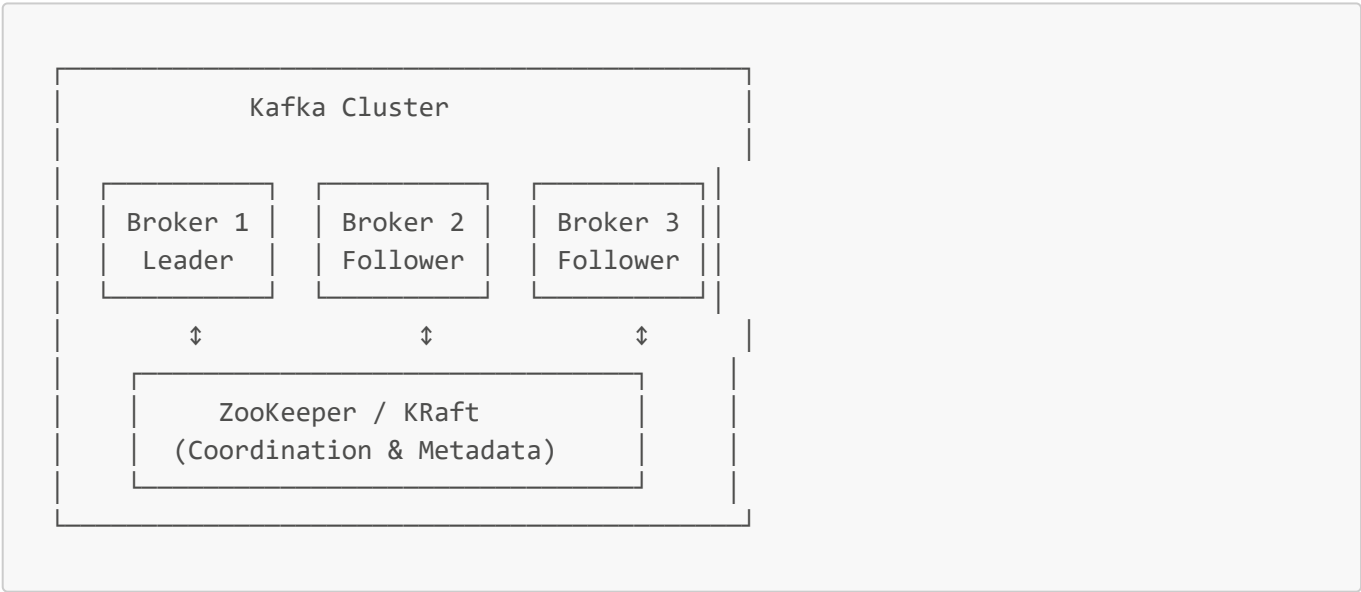
- 1. [Message Brokers](#)
- 2. [Stream Processors](#)
- 3. [Workflow Orchestration](#)
- 4. [Data Integration Tools](#)
- 5. [Cloud Services](#)
- 6. [Selection Framework](#)

1. Message Brokers {#message-brokers}

Apache Kafka

Overview: Distributed event streaming platform designed for high-throughput, fault-tolerant messaging.

Architecture:



Key Features:

- **Throughput:** Millions of messages/second
- **Latency:** 2-10ms (median)
- **Retention:** Configurable (days to forever)
- **Ordering:** Per-partition ordering guarantee
- **Durability:** Replication factor configurable

Performance Characteristics:

--

Benchmark Results (3-broker cluster, replication=3):

- Write throughput: 2M messages/sec
- Read throughput: 3M messages/sec
- Message size: 1 KB
- Network: 10 Gbps
- Storage: SSD

Pros: ☒ Mature, battle-tested ecosystem ☒ High throughput and low latency ☒ Strong ordering guarantees ☒ Replay capability (time-travel) ☒ Large community and tooling ☒ Connect framework for integrations

Cons: ☒ Complex to operate (ZooKeeper dependency) ☒ No built-in message filtering ☒ Partition management overhead ☒ Steeper learning curve

Pricing (self-hosted):

- EC2 instances: ~\$500-2K/month (small cluster)
- Storage: ~\$100-500/month
- Network: ~\$50-200/month
- **Total:** ~\$650-2.7K/month

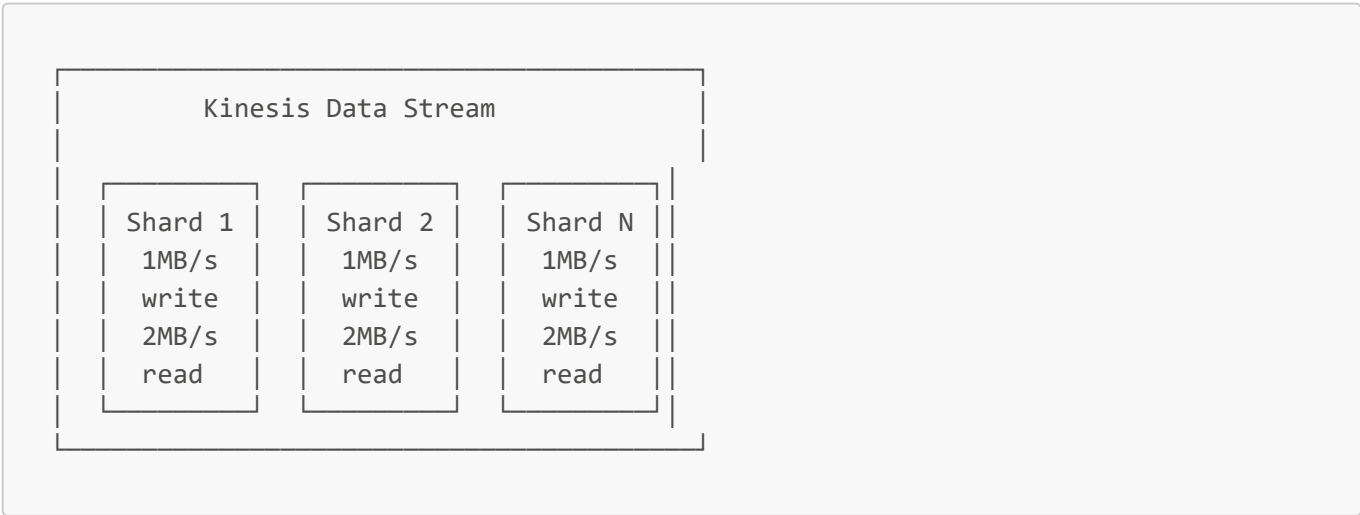
Use Cases:

- Event sourcing
- Stream processing pipelines
- Real-time analytics
- Log aggregation

AWS Kinesis

Overview: Fully managed streaming service on AWS.

Architecture:



Key Features:

- **Throughput:** 1 MB/s write, 2 MB/s read per shard
- **Latency:** ~70ms (typical)
- **Retention:** 24 hours to 365 days
- **Scaling:** Manual or automatic
- **Integration:** Native AWS integration

Performance Characteristics:

Per Shard:

- Write: 1,000 records/sec OR 1 MB/sec (whichever first)
- Read: 5 transactions/sec (up to 2 MB/sec)
- Message size: Max 1 MB

Scaling:

- Add shards for more capacity
- Resharding takes minutes

- Pros:** ☒ Fully managed (no ops overhead) ☒ Tight AWS integration ☒ Auto-scaling available ☒ Built-in encryption ☒ Simple pricing model
- Cons:** ☒ More expensive at scale ☒ Shard management complexity ☒ 1 MB message size limit ☒ Vendor lock-in (AWS only) ☒ Less flexible than Kafka

Pricing:

- Shard hour: \$0.015/hour (\$11/shard/month)
- PUT payload: \$0.014 per million
- Extended retention: \$0.023 per GB-month
- **Example:** 10 shards = ~\$120/month + data charges

Use Cases:

- AWS-native architectures
- Click stream analysis
- IoT data ingestion
- Log and event collection

Comparison Matrix: Message Brokers

Feature	Kafka	Kinesis	Pulsar	RabbitMQ	GCP Pub/Sub
Throughput	★★★★★	★★★★	★★★★★	★★★	★★★★
Latency	2-10ms	~70ms	5-10ms	1-5ms	50-100ms
Ordering	Per-partition	Per-shard	Per-partition	Queue-level	With key
Retention	Unlimited	365 days	Unlimited	None (ack)	7 days
Ops Complexity	High	Low	High	Medium	Low

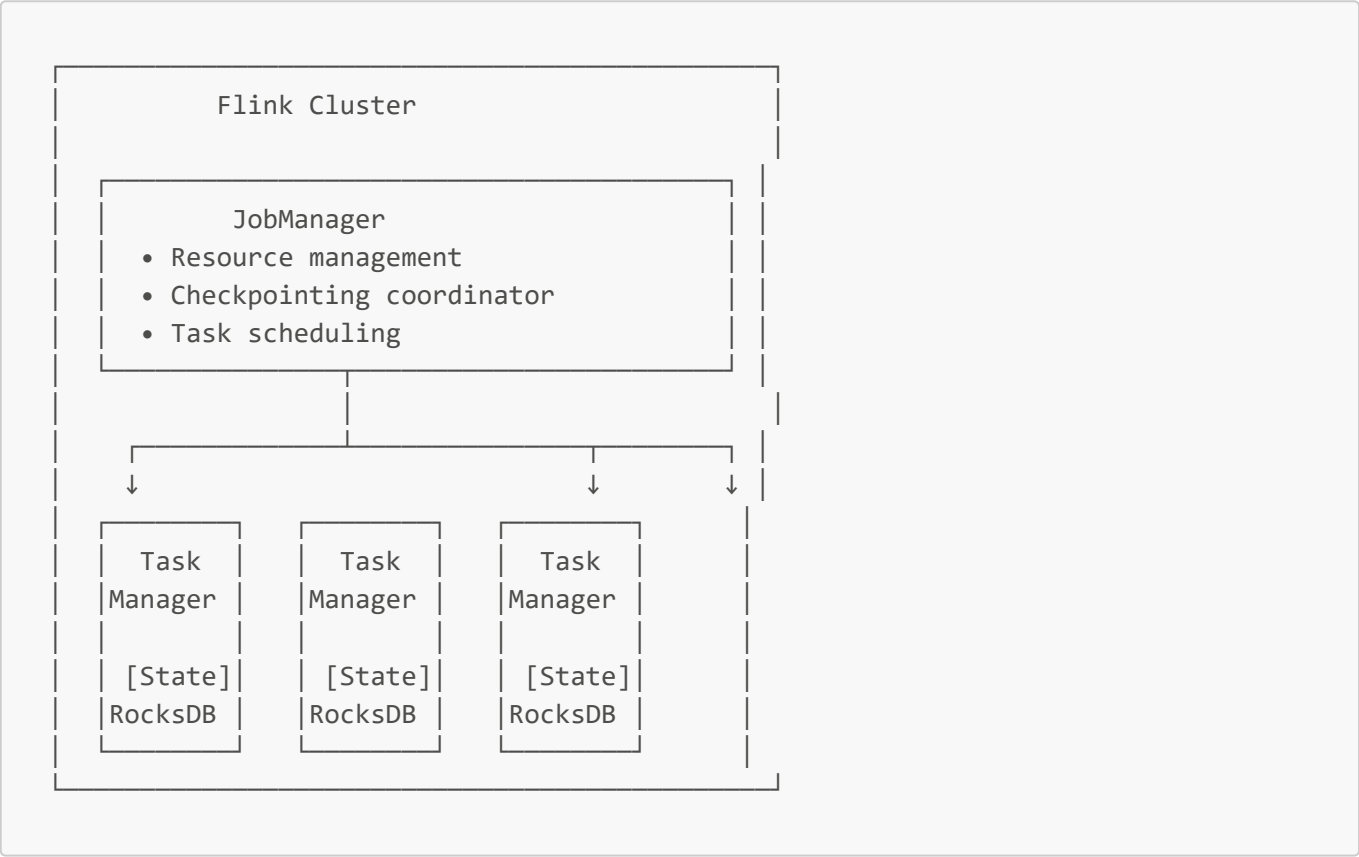
Feature	Kafka	Kinesis	Pulsar	RabbitMQ	GCP Pub/Sub
Cost (scale)	\$\$\$	\$\$\$\$	\$\$\$	\$\$	\$\$\$
Ecosystem	★★★★★	★★★★★	★★★	★★★★★	★★★★★

2. Stream Processors {#stream-processors}

Apache Flink

Overview: Distributed stream processing framework with true event-by-event processing.

Architecture:



Key Features:

- **Processing Model:** True streaming (event-by-event)
- **Latency:** <100ms
- **State Management:** Managed state with RocksDB
- **Fault Tolerance:** Exactly-once semantics
- **Windowing:** Event time, processing time, session windows
- **Scalability:** Horizontal scaling

Performance Characteristics:

Benchmark (8-node cluster):

- Throughput: 3M events/second
- Latency (p99): 60ms

- State size: TBs supported
- Checkpoint interval: 60 seconds
- Recovery time: <30 seconds

Code Example:

```
StreamExecutionEnvironment env =
    StreamExecutionEnvironment.getExecutionEnvironment();

env.enableCheckpointing(60000); // 60 second checkpoints

DataStream<Event> events = env
    .addSource(new KafkaSource("events"))
    .assignTimestampsAndWatermarks(...);

DataStream<Aggregate> results = events
    .keyBy(e -> e.userId)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .aggregate(new CountAggregator());

results.addSink(new ElasticsearchSink(...));

env.execute("Flink Stream Job");
```

Pros: ☒ True streaming (not micro-batch) ☒ Exactly-once guarantees ☒ Powerful state management ☒ Low latency (<100ms) ☒ Event time processing ☒ Rich windowing support ☒ Great for complex CEP

Cons: ☒ Steeper learning curve ☒ Requires cluster management ☒ More resource-intensive ☒ Smaller community than Spark ☒ Debugging can be complex

Resource Requirements:

Small deployment (dev/test):

- 1 JobManager: 2 vCPU, 4GB RAM
- 3 TaskManagers: 4 vCPU, 8GB RAM each
- Total: ~\$200-400/month

Medium deployment (production):

- 1 JobManager: 4 vCPU, 16GB RAM
- 10 TaskManagers: 8 vCPU, 32GB RAM each
- Total: ~\$2K-4K/month

Large deployment:

- 2 JobManagers (HA): 8 vCPU, 32GB RAM each
- 50 TaskManagers: 16 vCPU, 64GB RAM each
- Total: ~\$15K-25K/month

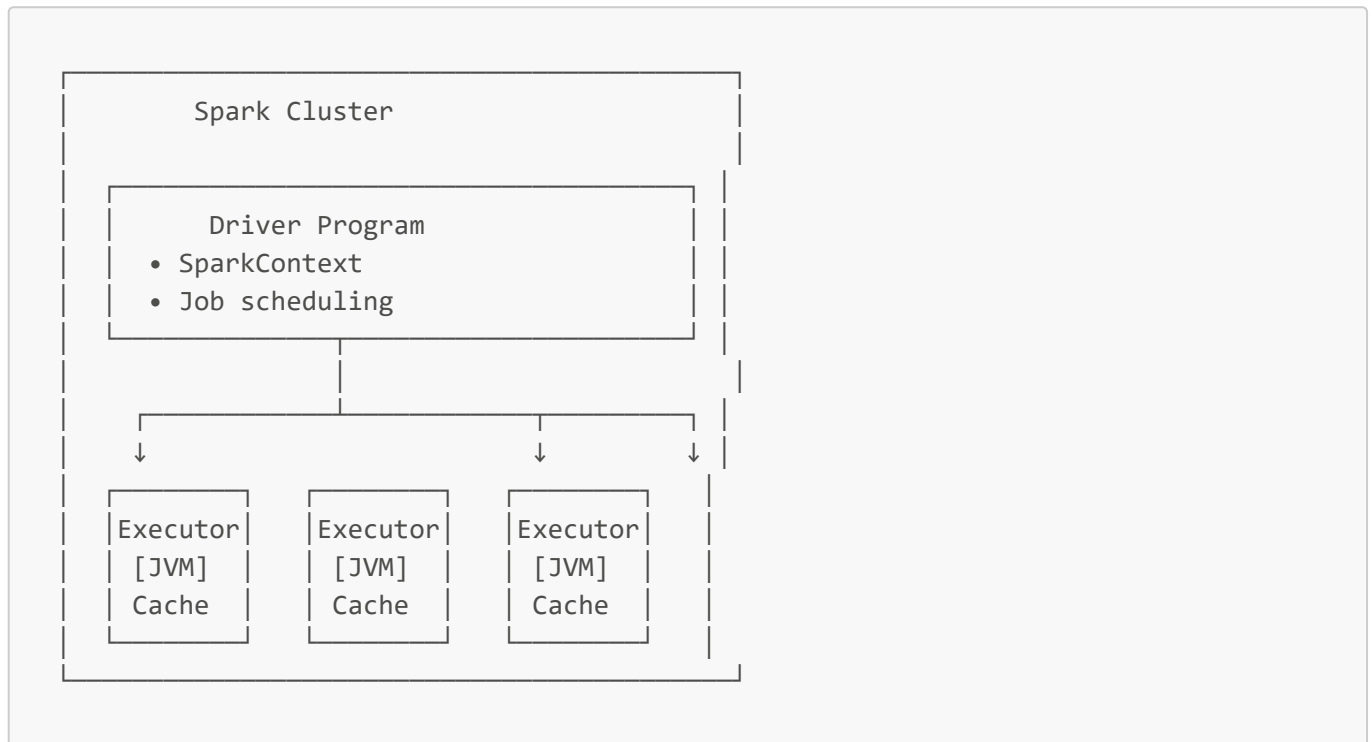
Use Cases:

- Real-time analytics
- Fraud detection
- Complex event processing
- Real-time ML inference

Apache Spark Structured Streaming

Overview: Micro-batch stream processing built on Spark batch engine.

Architecture:



Key Features:

- **Processing Model:** Micro-batching (1-2 second batches)
- **Latency:** 1-2 seconds (typical)
- **State Management:** Built-in stateful operations
- **Fault Tolerance:** Exactly-once with checkpointing
- **Integration:** Unified batch/stream API
- **Ecosystem:** Rich Spark ecosystem (MLlib, GraphX)

Performance Characteristics:

Benchmark (10-node cluster):

- Throughput: 1.5M events/second
- Latency (median): 1-2 seconds
- Micro-batch interval: 1 second
- Checkpoint interval: 10 seconds

Code Example:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder.appName("Streaming").getOrCreate()

events = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "events") \
    .load()

aggregated = events \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(
        window("timestamp", "5 minutes"),
        "user_id"
    ) \
    .count()

query = aggregated \
    .writeStream \
    .outputMode("update") \
    .format("console") \
    .start()

query.awaitTermination()
```

Pros: ☒ Easier learning curve (if know Spark) ☒ Unified batch/stream code ☒ Rich ecosystem (ML, GraphX)
☒ DataFrame API (SQL-like) ☒ Better for batch+stream ☒ More mature than Flink ☒ Great for ETL workloads

Cons: ☒ Higher latency (1-2s vs <100ms) ☒ Micro-batching not true streaming ☒ Less efficient state management ☒ More memory intensive ☒ Not ideal for ultra-low-latency

Resource Requirements:

Similar to Flink, slightly more memory-intensive

Use Cases:

- ETL pipelines
- Near real-time analytics
- ML feature engineering
- Data lake ingestion

Comparison: Stream Processors

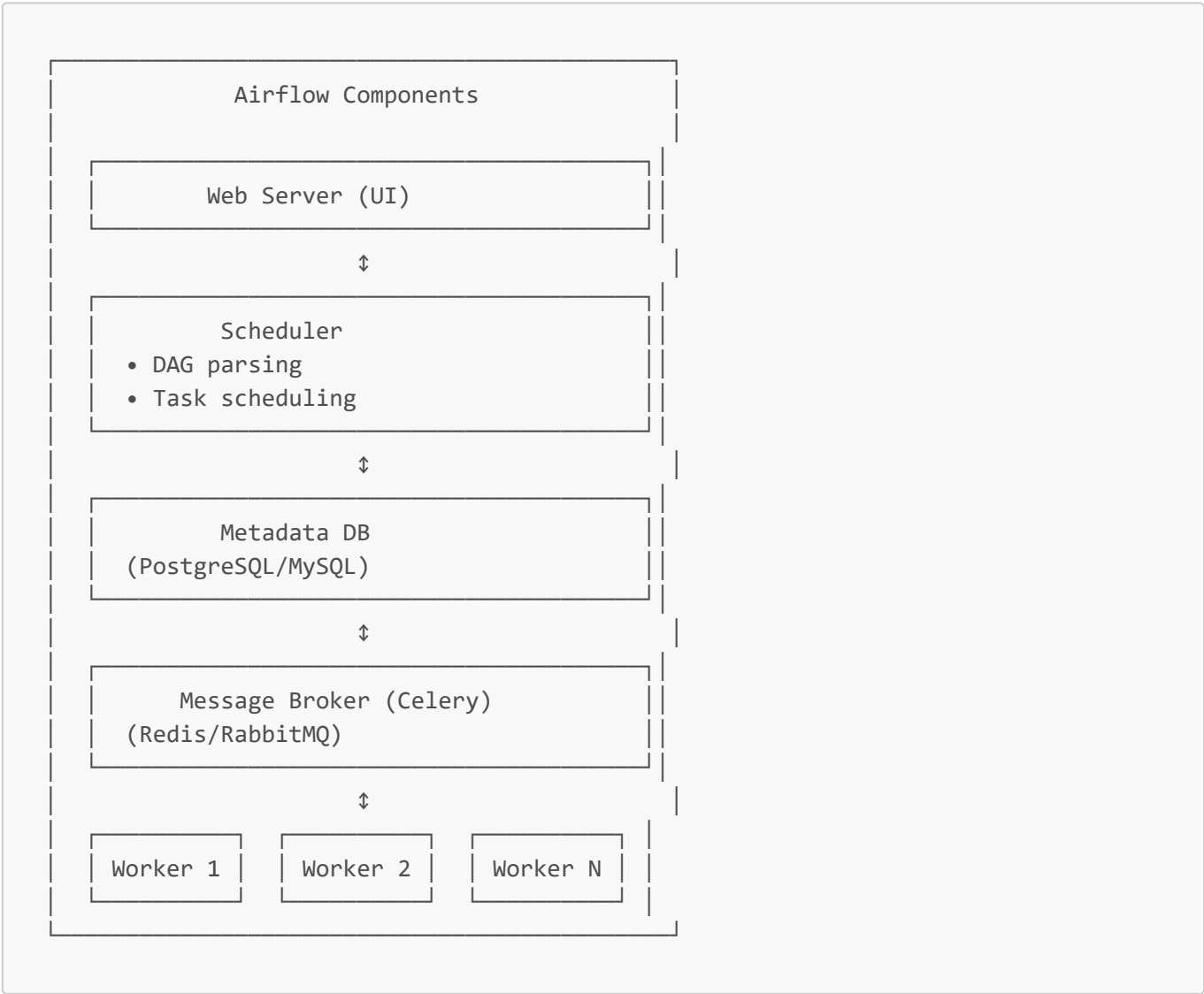
Feature	Flink	Spark Streaming	Kafka Streams	Storm
Processing	True streaming	Micro-batch	True streaming	True streaming
Latency	<100ms	1-2 seconds	<50ms	<100ms
Complexity	High	Medium	Low	Medium
State Mgmt	Excellent	Good	Good	Limited
Deployment	Cluster	Cluster	Embedded	Cluster
Ecosystem	Growing	Mature	Kafka-focused	Declining
Use Case	Complex CEP	Batch+Stream	Kafka apps	Simple streams

3. Workflow Orchestration {#workflow-orchestration}

Apache Airflow

Overview: Platform to programmatically author, schedule, and monitor workflows.

Architecture:



Key Features:

- **DAG-based:** Workflows as Directed Acyclic Graphs
- **Python-first:** Define workflows in Python
- **Rich UI:** Monitor and troubleshoot
- **Extensible:** 300+ operators/providers
- **Scheduling:** Cron-like scheduling
- **Backfill:** Reprocess historical data

Code Example:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.operators.s3 import S3CopyObjectOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-eng',
    'depends_on_past': False,
    'email_on_failure': True,
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    'data_pipeline',
    default_args=default_args,
    description='Daily data pipeline',
    schedule_interval='@daily',
    start_date=datetime(2024, 1, 1),
    catchup=False,
) as dag:

    extract = PythonOperator(
        task_id='extract_data',
        python_callable=extract_from_db
    )

    transform = PythonOperator(
        task_id='transform_data',
        python_callable=transform_data
    )

    load = S3CopyObjectOperator(
        task_id='load_to_s3',
        source_bucket='staging',
        dest_bucket='production'
    )

    extract >> transform >> load
```

Pros: ☒ Python-native (easy for data engineers) ☒ Rich UI for monitoring ☒ Large ecosystem of operators ☒ Strong community ☒ Battle-tested ☒ Good for complex dependencies

Cons: ☒ Not ideal for real-time ☒ Scheduler can be bottleneck ☒ DAG serialization overhead ☒ Steep learning curve ☒ Requires infrastructure ☒ Can be resource-heavy

Resource Requirements:

Small (dev):

- Web server: 2 vCPU, 4GB
- Scheduler: 2 vCPU, 4GB
- DB: 2 vCPU, 4GB
- Workers: 2-4 workers (4 vCPU, 8GB each)

Total: ~\$300-600/month

Large (production):

- Web servers: 2x (4 vCPU, 8GB)
- Schedulers: 2x (4 vCPU, 16GB)
- DB: RDS (4 vCPU, 16GB)
- Workers: 20x (8 vCPU, 16GB)

Total: ~\$4K-8K/month

Use Cases:

- Batch ETL pipelines
- Data warehouse loading
- ML model training
- Report generation

Comparison: Orchestration Tools

Feature	Airflow	Prefect	Dagster	AWS Step Functions
Programming	Python	Python	Python	JSON (states)
Deployment	Self-hosted	Cloud/Self	Self-hosted	AWS managed
UI	Excellent	Good	Excellent	Basic
Real-time	No	Limited	Limited	No
Complexity	Medium	Low	Medium	Low
Cost	\$\$	\$\$\$	\$\$	\$
Best For	Complex ETL	Modern pipelines	Data apps	AWS workflows

4. Data Integration Tools {#data-integration}

Debezium (CDC)

Overview: Open-source CDC platform for capturing database changes.

Supported Databases:

- MySQL, PostgreSQL, MongoDB
- SQL Server, Oracle, Db2
- Cassandra, Vitess

Deployment:

```
Option 1: Kafka Connect
  Debezium → Kafka Connect → Kafka

Option 2: Debezium Server (standalone)
  Debezium Server → Kinesis/Pulsar/HTTP

Option 3: Embedded
  Your App + Debezium Engine
```

Pros: ☒ Open-source, no licensing ☒ Real-time change capture ☒ Multiple database support ☒ Rich event format ☒ Schema evolution support

Cons: ☒ Requires database config ☒ Operational overhead ☒ Learning curve ☒ Per-database connector differences

Airbyte

Overview: Open-source data integration platform (ETL/ELT).

Features:

- **Connectors:** 300+ sources and destinations
- **CDC Support:** For select sources
- **Scheduling:** Built-in scheduling
- **Transformations:** Basic (dbt integration for advanced)
- **Deployment:** Self-hosted or cloud

Pros: ☒ Easy to use (UI-driven) ☒ Many pre-built connectors ☒ Open-source ☒ Active development ☒ Modern architecture

Cons: ☒ Less mature than Fivetran ☒ Some connectors limited ☒ Performance varies by connector ☒ Self-hosting requires ops

Pricing (Cloud):

- Free tier: 1 million rows/month
- Team: \$250/month + usage
- Enterprise: Custom pricing

Comparison: Data Integration

Feature	Debezium	Airbyte	Fivetran	Stitch
Focus	CDC only	All sources	All sources	All sources
Connectors	10 (DBs)	300+	150+	130+
Pricing	Free (OSS)	Free/Paid	\$\$\$	\$\$
Ease of Use	Low	High	Highest	High
Customization	High	Medium	Low	Low
Real-time	Yes	Limited	Limited	No

5. Cloud Services {#cloud-services}

AWS

Key Services:

- **Kinesis**: Streaming (alternative to Kafka)
- **MSK**: Managed Kafka
- **Glue**: Serverless ETL
- **EMR**: Managed Spark/Flink
- **DMS**: Database migration
- **EventBridge**: Event bus

Azure

Key Services:

- **Event Hubs**: Streaming (Kafka-compatible)
- **Stream Analytics**: Stream processing (SQL-based)
- **Data Factory**: ETL orchestration
- **Synapse**: Analytics platform
- **HDInsight**: Managed Hadoop/Spark

GCP

Key Services:

- **Pub/Sub**: Messaging
- **Dataflow**: Unified stream/batch (Apache Beam)
- **Dataproc**: Managed Spark
- **Composer**: Managed Airflow
- **Data Fusion**: Visual ETL

6. Selection Framework {#selection-framework}

Decision Tree

1. Latency Requirement?
 - └ <100ms → Flink + Kafka
 - └ 1-5 seconds → Spark Streaming + Kafka
 - └ Minutes+ → Airflow + Batch
2. Source Type?
 - └ Databases → Debezium (CDC)
 - └ APIs → Airbyte / Custom
 - └ Files → Cloud services (S3 events, etc.)
3. Scale?
 - └ <1M events/day → Managed services (Kinesis)
 - └ 1M-100M → Kafka + Spark
 - └ 100M+ → Kafka + Flink
4. Team Expertise?
 - └ Python-heavy → Airflow + Spark (PySpark)
 - └ Java-heavy → Flink + Kafka
 - └ SQL-focused → dbt + Cloud warehouse

Cost Optimization

Rule of Thumb:

- <\$50K/year: Managed services (Kinesis, Cloud Functions)
- \$50K-500K/year: Hybrid (MSK + managed compute)
- \$500K+/year: Self-hosted (EC2/EKS + Kafka + Flink)

Conclusion

No single tool fits all use cases. Key considerations:

1. **Requirements First:** Latency, scale, complexity
 2. **Team Skills:** Work with your team's strengths
 3. **Cost:** Factor in ops overhead, not just licenses
 4. **Ecosystem:** Choose technologies that work together
 5. **Future:** Plan for growth and evolution
-

Version: 1.0